Name : UDHAYARAJAN J.
Class : MCA - I.
Subject : DataStructure using java.
Date : 2026-04-06.
Type : CCF - 03 (Trace the output).
Roll no : 2202561.

What will be the output of this code. Show diagrammatic presentation as well.

① Consider linear stack.

Stack → Last In First Out (LIFO) principle.

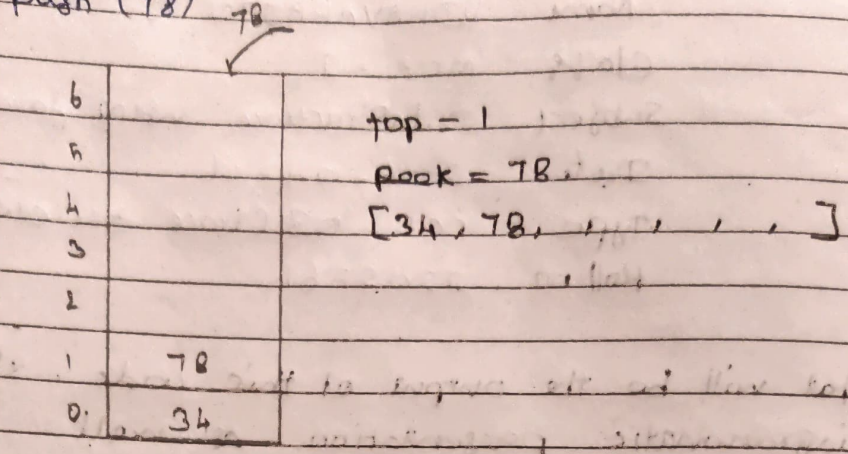
→ int arr[7];

6		size = 7.
5		top = -1
4		peek = -1
3		
2		
1		
0		

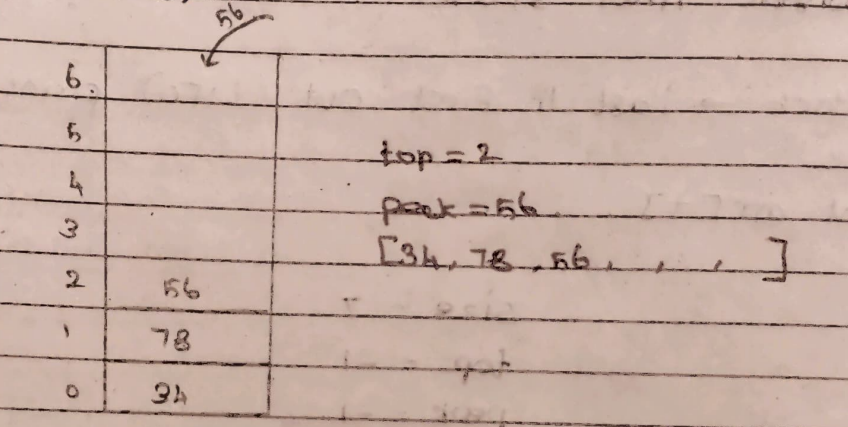
→ push(34).

6		
5		top = 0.
4		peek = 34.
3		[34, , , , , ,]
2		
1		
0	34	

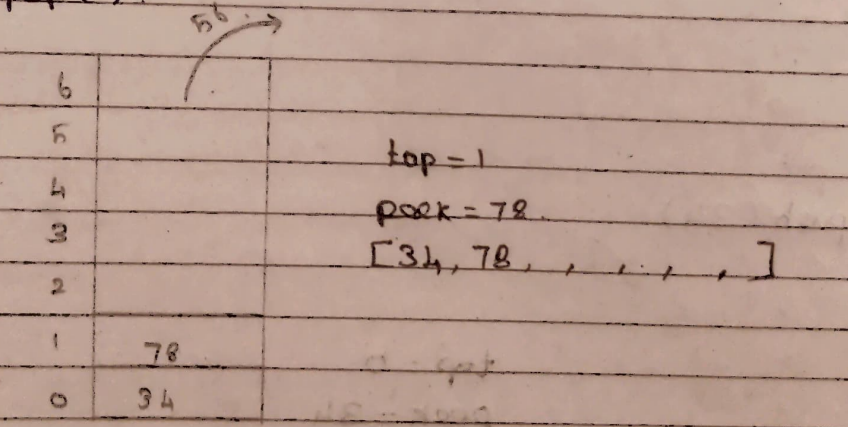
→ push (78)



→ push (56)



→ pop()



→ push (67)

6	
5	
4	
3	
2	67
1	78
0	34

top = 2
peek = 67.
[34, 78, 67, , , ,].

→ push(51).

6	
5	
4	
3	51
2	67
1	78
0	34

top = 3.
peek = 51
[34, 78, 67, 51, , ,].

→ peek ( ) peek = 51

→ pop ( )

6.	
5	
4	
3	
2	67
1	78
0	34

top = 2
peek = 67.
[34, 78, 67, , , ,].

→ pop( ).

6	
5	
4	
3	
2	
1	78
0	34

top = 1
peek = 78
[34, 78, , , , ,]

→ push(39)

6	
5	
4	
3	
2	39
1	78
0	34

top = 2
peek = 39
[34, 78, 39, , , ,]

→ pop( )

6	
5	
4	
3	
2	
1	78
0	34

top = 1
peek = 78
[34, 78, , , , ,]

→ pop( )

6	
5	
4	
3	
2	
1	
0	34

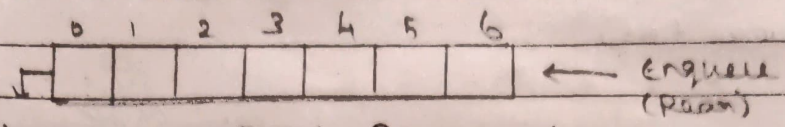
output of this code

top = 0
peek = 34
[34, , , , , ,]

② Consider Linear Queue:

Queue → First In First Out.

→ int arr[7].

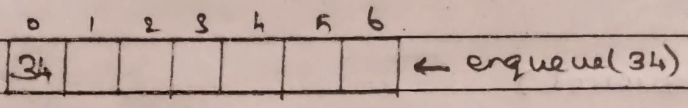


Front, Rear = -1

Peek = -1

Size = 7.

→ enQueue(34)

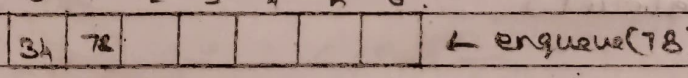


Front = 0

Rear = 0

Peek = 34.

→ enqueue (78)

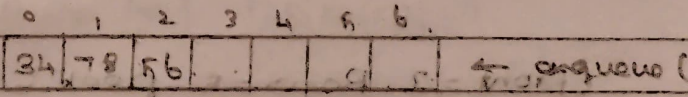


Front = 0.

Rear = 1

peek = 34.

→ enqueue (56).



Front = 0.

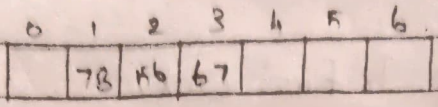
Rear = 2

Peek = 34.

→ dequeue().

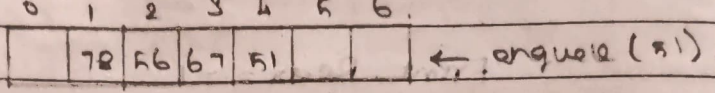
(Front = 1, Rear = 2, peek = 78)

→ enQueue(67)



(Front = 1, Rear = 3, peek = 78)

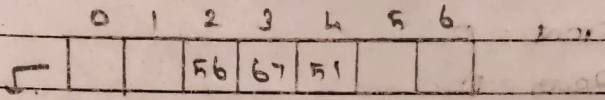
→ enQueue(51)



(Front = 1, Rear = 4, peek = 78)

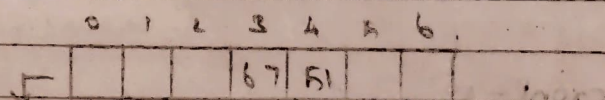
→ peek() peek = 78

→ dequeue()



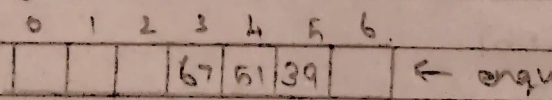
dequeue (Front = 2, Rear = 4, peek = 56)
(-78)

→ dequeue()



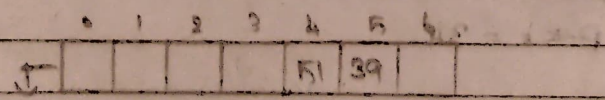
dequeue (56) (Front = 3, Rear = 4, peek = 67)

→ enqueue(39)



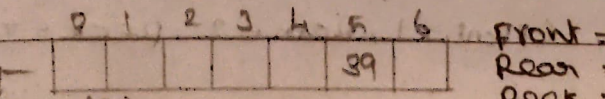
(Front = 3, Rear = 5, peek = 67)

→ dequeue()



dequeue (67) (Front = 4, Rear = 5, peek = 51)

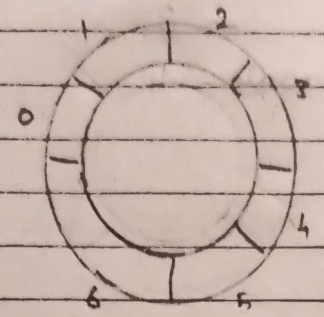
→ dequeue()



Front = 5
Rear = 5
peek = 39.

③ Consider Circular Queue

→ int arr[7];

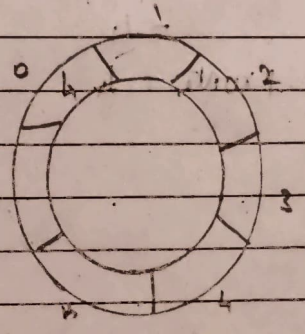


Front = -1

Rear = -1

[, , , , , ,]

→ enqueue(4);



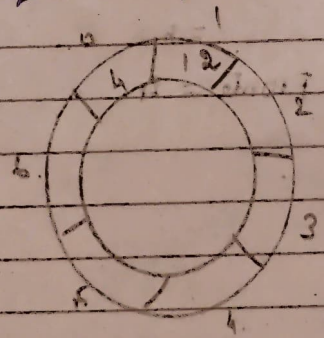
Front = 0

Rear = 0

[4, , , , , ,]

Front = 4

→ enqueue(12)



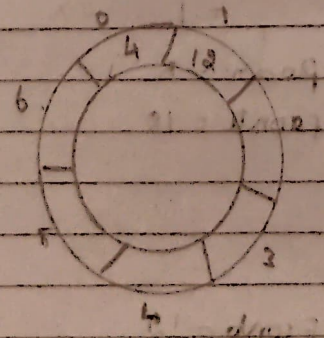
Front = 0

Rear = (0+1) % 7 => 1 % 7

= 1

Front = 4

→ enqueue(13);



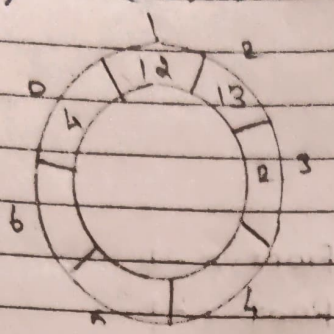
Front = 0

Rear = (1+1) % 7 => 2 % 7

= 2

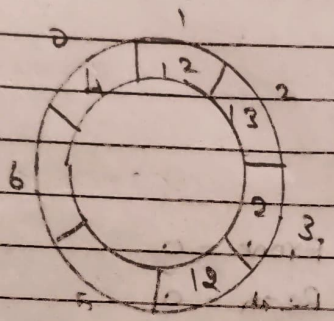
Front = 4

→ enqueue (2);



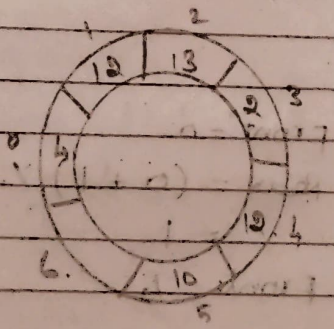
Front = 0.
Rear = (3+1) % 7 = 3 % 7
= 3.
Fronte = 4.

→ enqueue (12)



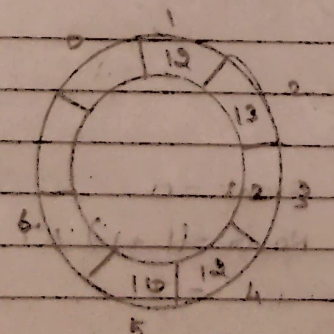
Front = 0.
Rear = (3+1) % 7 = 4 % 7
= 4.
Fronte = 4.

→ enqueue (10)



Front = 0.
Rear = (4+1) % 7 = 5 % 7
= 5
Fronte = 4.

→ dequeue().

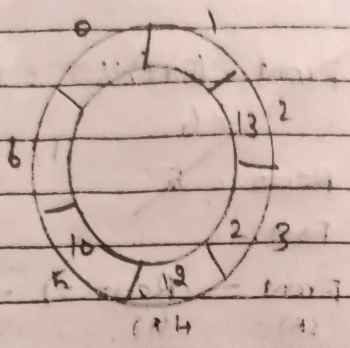


Front = (0+1) % 7 = 1 % 7
= 1
Rear = 5
Fronte = 12

→ fronte ()

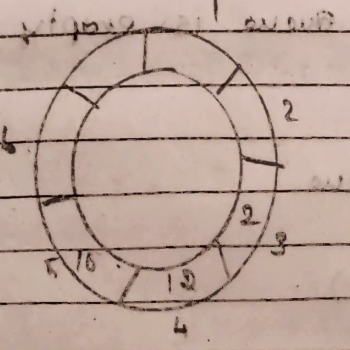
Fronte = 12.

→ dequeue ( ).



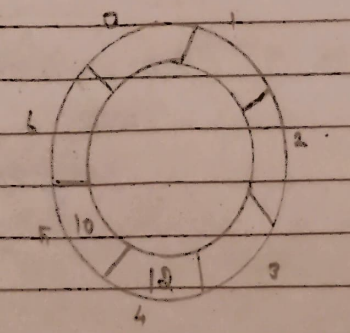
Front = (1+1) % 7 = 2 % 7
= 2.
Rear = 5.
Fronte = 13.

→ dequeue ( ).



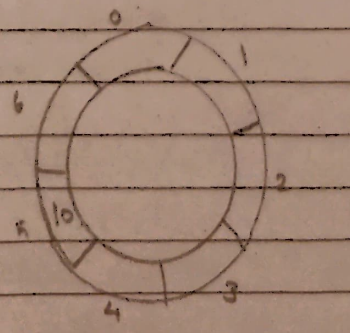
Front = (2+1) % 7 = 3 % 7
= 3.
Rear = 5
Fronte = 8.

→ dequeue ( )



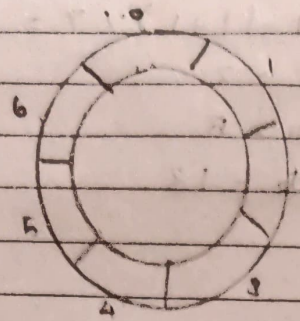
front = (3+1) % 7 = 4 % 7
= 4.
Rear = 5
Fronte = 12

→ dequeue ( )



Front = (4+1) % 7 = 5 % 7
= 5
Rear = 5
Fronte = 10

→ dequeue ( )



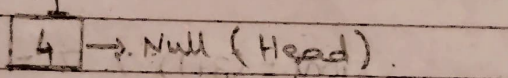
Front = (5+1) % 7 = 6 % 7
= 6.
Rear = 5
Front
Front == Rear ⇒ -1.
(5) (5)

→ display ( )

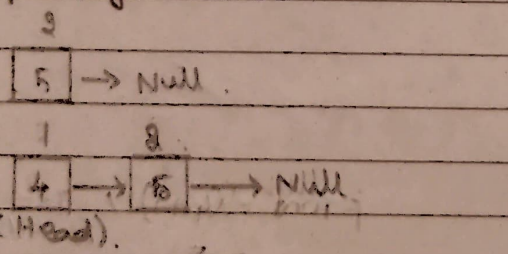
Queue is empty.

(4) Consider priority Queue.

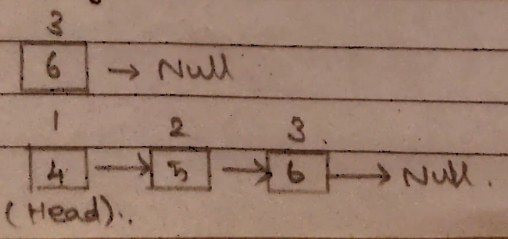
→ var pq = new Node(4, 1).
data = 4.
priority = 1.



→ pq = push(pq, 5, 2).
data = 5
priority = 2.



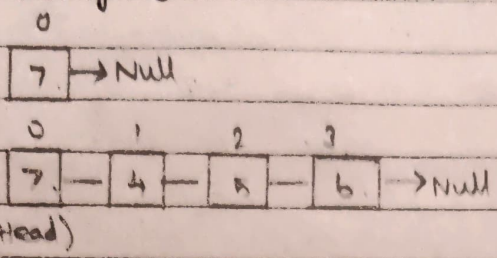
→ pq = push(pq, 6, 3)
data = 6.
priority = 3.



→ pq = push(pq, 7, 0)

data = 7.

priority = 0.



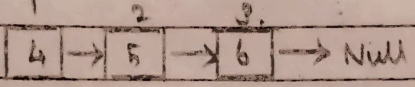
→ while (isEmpty(pq)) == 0).

① (head == null) ?1:0 ⇒ 0 // ✓.

peek = 7

pop(pq).

head = head.next.

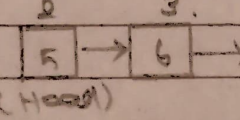


② (head == null) ?1:0 ⇒ 0 // ✓.

peek = 4.

pop(pq).

head = head.next.

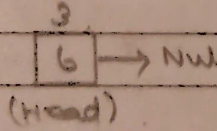


③ (head == null) ?1:0 ⇒ 0 // ✓.

peek = 5.

pop(pq)

head = head.next.



④ (head == null) ?1:0 ⇒ 0 // ✓.

peek = 6.

pop(pq).

head = null.

⑤ ((head) == null) ?1:0 ⇒ 1 //(X).